BHARATIYA VIDYA BHAVAN'S
SARDAR PATEL INSTITUTE OF TECHNOLOGY

Bhavan's Campus, Munshi Nagar, Andheri (West), Mumbai – 400058
DEPARTMENT OF COMPUTER ENGINEERING
SUBJECT: GENERATIVE AI

Student Information	Details
Name	Raj Mathuria
UID No.	2023300139
Batch	PE - C
Experiment No.	4
Assigned Modification	Modification 3 — Trajectory-Level Preferences
Repository	https://github.com/CodeCraftsmanRaj/GenAI/tree/main/Exp4

EXPERIMENT 4: REWARD MODELING AND REINFORCEMENT LEARNING

AIM

To build a preference-based reward predictor and use its learned reward signal to train a Q-learning agent in a GridWorld. The assigned extension, Modification 3, compares complete trajectories instead of judging each action in isolation.

THEORY

1. Reinforcement Learning

Reinforcement learning (RL) trains an agent through repeated interaction with an environment. At each step, the agent observes a state, chooses an action, and receives feedback. The objective is to learn a policy that maximizes the sum of rewards over time.

The main elements are the state (the agent's current situation), action (a possible move), reward (numerical feedback), environment (the world in which the agent operates), policy (the action-selection strategy), and Q-value (the estimated long-term value of an action in a state).

2. GridWorld Environment

The experiment uses a 4 × 4 grid containing a starting cell, a target cell, free cells, and an obstacle. The agent can move up, down, left, or right. The environment assigns −1 for a normal move, −5 for an attempted move into an obstacle, and +10 for reaching the goal. A useful policy should reach the target without wasting moves or repeatedly hitting the obstacle.

```
S  .  .  .
.  X  .  .
.  .  .  .
.  .  .  G
```

Here, S denotes the start, G marks the goal, X is blocked, and a dot indicates an available cell.

3. Q-Learning

Q-learning is a model-free method that estimates the value of taking an action in a particular state. After an action, the Q-table is updated using the observed reward and the best estimated value of the next state:

$$Q(s,a) \leftarrow Q(s,a) + \alpha\,[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

In this expression, α is the learning rate, γ discounts future rewards, r is the received reward, s′ is the next state, and a′ is a possible next action. In this experiment, the agent uses the reward predicted by the learned Reward Model for its Q-learning updates rather than relying only on the environment's hand-coded reward.

4. Reward Modeling from Preferences

Reward modeling learns a scoring function from comparisons between alternatives. Instead of requiring an exact reward label for every example, the training data identifies which of two alternatives is preferred. The model is optimized to assign a higher score to the preferred alternative than to the rejected one. In this setup, preference labels are simulated using rules that reflect progress toward the goal, obstacle avoidance, and path efficiency.

5. Pairwise Preference Learning

For a preferred item with predicted reward r_v and a rejected item with predicted reward r_l, the model aims for $r_v > r_l$. A Bradley–Terry-style formulation converts their reward difference into a preference probability:

$$P(\text{preferred}) = \sigma(r_v - r_l), \quad \sigma(x) = 1 / (1 + e^{-x})$$

The corresponding negative log-sigmoid loss penalizes the model when it assigns too little relative reward to the preferred example. Training should therefore reduce the loss as the model learns the ranking expressed by the preference pairs.

6. Modification 3 — Trajectory-Level Preferences

The main modification changes the comparison unit from a single state-action pair to a complete short sequence of moves. Two candidate trajectories begin from the same initial state, then are compared as whole paths. This is useful because a move that looks unhelpful on its own—for example, temporarily moving away from the goal to go around an obstacle—may contribute to a better overall route.

The supplied experiment scores a trajectory using goal progress, the number of obstacle attempts, and its length:

$$S(\tau) = 3P_g - 4N_o - 0.5L$$

Here, P_g represents progress toward the goal, N_o counts obstacle attempts, and L is the number of steps. A higher score marks the preferred trajectory. The Reward Model processes the sequence of state-action pairs, predicts a reward for each step, and sums the step rewards to obtain the total trajectory reward:

$$R(\tau) = \Sigma_{t=1}^{T} r_t$$

7. Exploration, Exploitation, and Proxy-Reward Limitations

An ε-greedy policy balances exploration and exploitation: with probability ε the agent chooses a random action, while otherwise it selects the action with the largest Q-value. Reducing ε during training gradually shifts behavior toward using what has been learned.

The learned reward is a proxy for the intended objective, not necessarily the true environment reward. A model may learn patterns in the preference data that do not reliably lead to goal completion. This is often described as reward-model failure or reward hacking. Therefore, preference-prediction accuracy and downstream RL behavior should be evaluated separately.

ALGORITHM / PROCEDURE

1. Create the GridWorld, define the starting state, goal, obstacle, action set, and environment reward rules.
2. Generate candidate trajectories from common starting states and calculate preference scores using goal progress, obstacle attempts, and path length.
3. Construct preferred/rejected trajectory pairs to form simulated preference data.
4. Represent each state-action pair numerically and train a neural Reward Model using a pairwise preference loss.
5. Aggregate predicted step rewards to obtain a score for each trajectory.
6. Use the learned reward as the reward signal for a Q-learning agent and select actions with an ε-greedy policy.
7. Track the Reward Model training loss, test preference accuracy, and the RL agent's cumulative reward, path length, and goal-reaching behavior.

RESULTS AND OBSERVATIONS

The results below retain the reported values and figures from the supplied Experiment 4 report. The assigned modification generated 3,000 trajectory-preference examples, with 2,400 used for training and 600 for testing.

Reported item	Result
Trajectory preference dataset	3,000 examples
Training preference pairs	2,400
Testing preference pairs	600
Test preference accuracy	Approximately 85.17%
Reward Model	Functional neural-network model

Reward Model Training

The plotted preference loss declines steadily over training, from roughly 0.69 at the start to around 0.15 near the final epoch. This downward trend suggests that the model increasingly fits the preference comparisons in the training data.

```
Reward Model Epoch  20 Loss = 0.4881
Reward Model Epoch  30 Loss = 0.4013
Reward Model Epoch  40 Loss = 0.3235
Reward Model Epoch  50 Loss = 0.2616
Reward Model Epoch  60 Loss = 0.2170
Reward Model Epoch  70 Loss = 0.1874
Reward Model Epoch  80 Loss = 0.1680
Reward Model Epoch  90 Loss = 0.1546
Reward Model Epoch 100 Loss = 0.1449
```

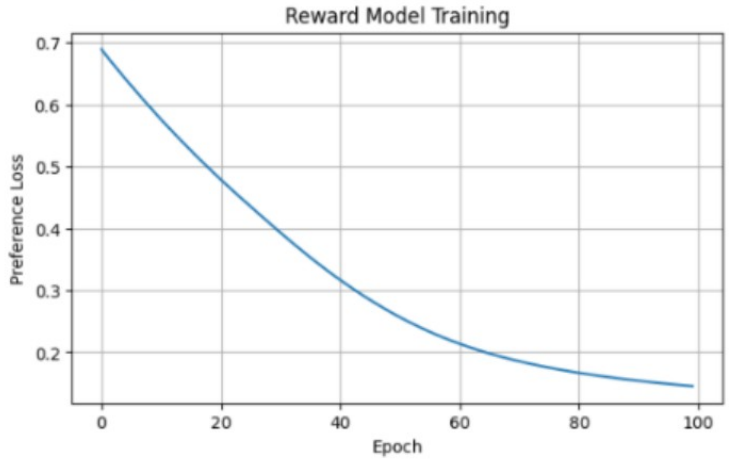
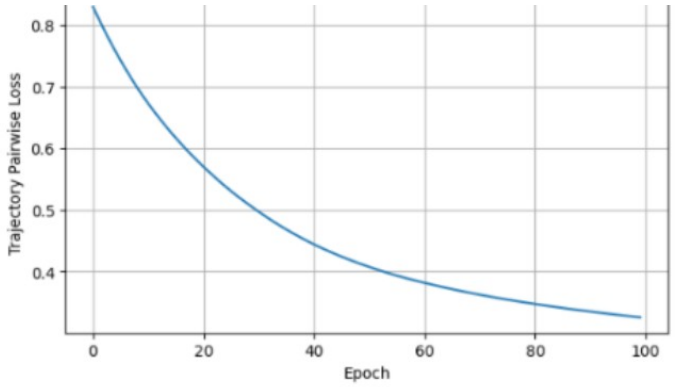


Figure 1. Reward Model training output and loss curve reproduced from the supplied report.

Trajectory-Level Preference Examples

The output lists candidate trajectories, their computed scores, and the selected preferred trajectory. These examples demonstrate the use of sequence-level comparisons: the preferred path is selected using the overall score rather than by judging each move separately.




```
====================================
TRAJECTORY PREFERENCE ACCURACY
====================================
Training Accuracy: 0.8541666666666666
Testing Accuracy : 0.8516666666666667
Episode  100 | Reward =   -2.62 | Steps =   9 | Success = 1 | Epsilon = 0.606
Episode  200 | Reward =   -0.09 | Steps =  11 | Success = 1 | Epsilon = 0.367
Episode  300 | Reward =   -3.56 | Steps =  19 | Success = 1 | Epsilon = 0.222
Episode  400 | Reward =   -1.66 | Steps =  30 | Success = 1 | Epsilon = 0.135
Episode  500 | Reward =    1.06 | Steps =  17 | Success = 1 | Epsilon = 0.082
Episode  600 | Reward =    3.72 | Steps =  42 | Success = 1 | Epsilon = 0.050
Episode  700 | Reward =    3.20 | Steps =  31 | Success = 1 | Epsilon = 0.050
Episode  800 | Reward =   10.01 | Steps = 100 | Success = 0 | Epsilon = 0.050
Episode  900 | Reward =    4.47 | Steps =  47 | Success = 1 | Epsilon = 0.050
Episode 1000 | Reward =    0.43 | Steps =  28 | Success = 1 | Epsilon = 0.050
Episode 1100 | Reward =   -2.35 | Steps =  11 | Success = 1 | Epsilon = 0.050
Episode 1200 | Reward =    4.12 | Steps =  37 | Success = 1 | Epsilon = 0.050
```

Figure 2. Trajectory-preference examples and model-training details from the supplied report.

Reinforcement Learning Performance

The supplied learning plot shows the agent's cumulative return across episodes. The trend improves from strongly negative values early in training toward more favorable returns later, although the curve remains variable. This indicates learning progress but also shows why the final policy should be checked for reliable goal completion rather than relying only on reward-model accuracy.

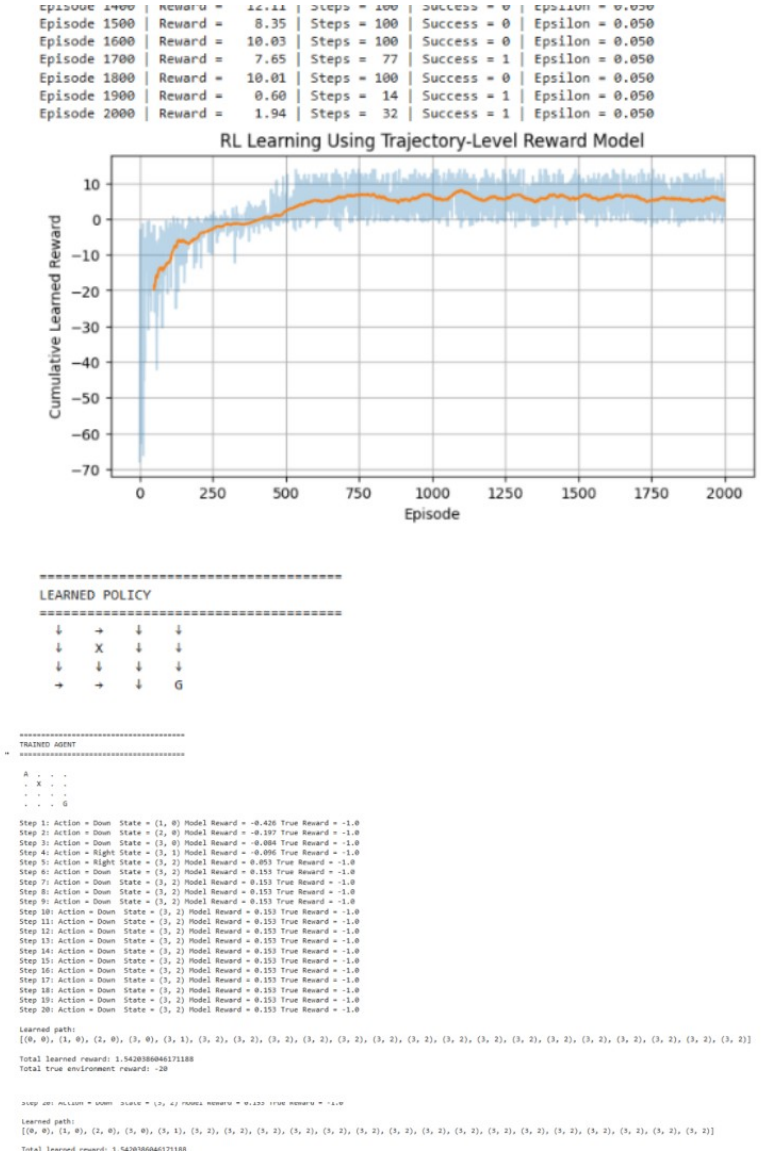


Figure 3. RL learning curve and policy output from the supplied report.

Discussion

The reported test preference accuracy of 85.17% indicates that the Reward Model correctly ranks most held-out trajectory pairs. However, this metric alone cannot establish that the Q-learning policy consistently reaches the goal. The trajectory-level modification provides a more context-aware comparison than isolated action preferences, while the proxy-reward limitation remains important when interpreting the final behavior.

CONCLUSION

This experiment explored preference-based reward learning and reinforcement learning in a 4 × 4 GridWorld. A neural Reward Model was trained on simulated preference pairs and then used to provide the reward signal for a Q-learning agent. Modification 3 compared complete trajectories, using progress toward the target, obstacle avoidance, and path length to determine which route was preferable. The supplied results report approximately 85.17% test preference accuracy and a decreasing training-loss curve. The experiment also highlights that successful preference prediction does not automatically ensure an optimal RL policy, because the learned reward is only an approximation of the intended objective.

GITHUB REPOSITORY

https://github.com/CodeCraftsmanRaj/GenAI/tree/main/Exp4